

Name : AKASH SAMPATH

Module : Week-1

1. Client Server Model :

In client server model we have client – server – request – response

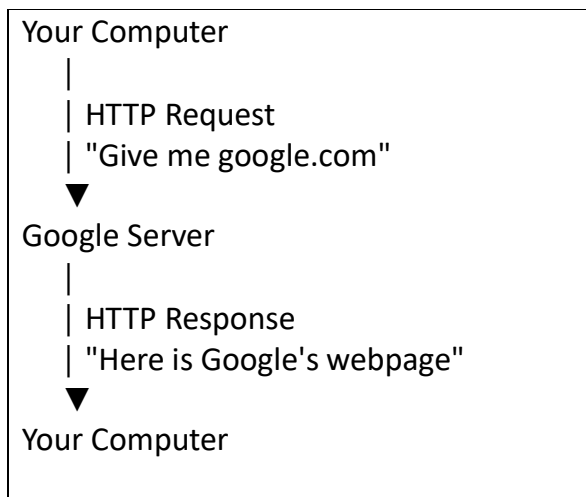
So the client asks the sever by request and based on the request by the server from client we get the response

So basically the client starts the conversation and the server responds

Suppose you open chrome and type : <https://www.google.com>

Your computer is acting as the client and google computer system acts as the server and chrome then displays the response as a webpage

Chrome	Client
Google server	Server
HTTP / HTTPS	Communication Language or Protocol
Request	What we ask for
Response	What server sends back



HTTP :

This stands for Hypertext Transfer Protocol and it is a set of rules for communication

You client says GET /users and server understands the client wants me to send the users

So HTTP provides a standard language that both sides such as client and server understands

```
[
  {
    "id": 1,
    "name": "Akash"
  },
  {
    "id": 2,
    "name": "Rahul"
  }
]
```

Request :

A request is the message sent by the client to the server

So GET `https://example.com/users`

The client is saying server please give users list and a request can contain things such as method as HTTP or HTTPS , URL , Headers and Body

```
POST /users
Content-Type: application/json

{
  "name": "Akash",
  "email": "akash@example.com"
}
```

So server creates a new user with this information

Response :

The server processes the request and sends a response and therefore the server is saying that it successfully created the user with new users information

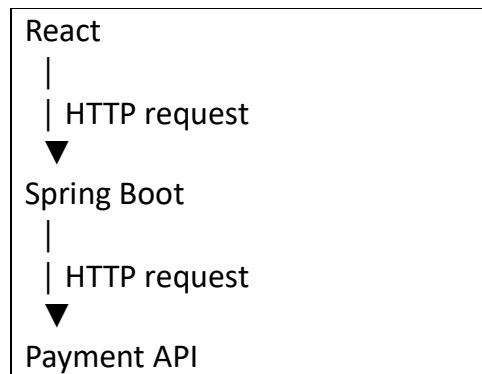
Is the browser not the server :

So the chrome is the client and amazon infra is the server side and our browser such as chrome doesn't contain Amazon's database

So the browser asks amazon by give me this product and amazon backend process the request and sends the information back

Is a server can also be a client :

So react application is a client of your spring boot backend and also spring boot backend can also become a client when it communicate with another service such as Payment API



So first the react is the client where it sends the HTTP request to spring boot which is the server and second spring boot is the client where gets HTTP requests from react which was the client and finally from spring boot which is the client sends HTTP request to the server which is the payment provider

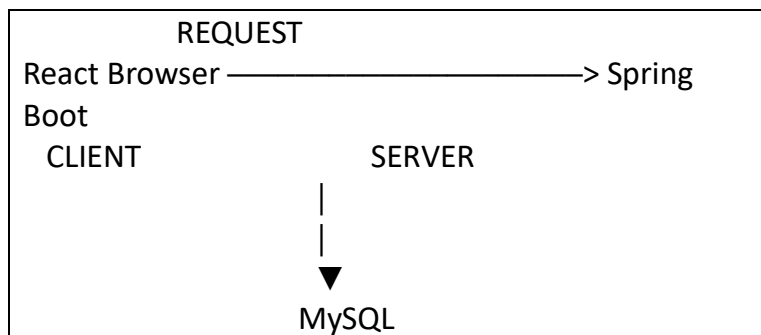
So client and server are roles

Electricity Bill Project :

So we have React , Spring boot and MYSQL

So the user opens the application using <https://electricity.com> and react frontend appears and the user clicks view bill and react needs the bill information and so react sends GET /api/bills/101 to your spring boot backend

Finally spring boot asks MYSQL for the bill and MYSQL returns the data and spring boot then sends a response back and browser displays the details



AWS Understanding :

User's Browser	Client
Cloudfront	Receives HTTP / HTTPS requests from client
S3	Stores you react build files
ALB (Application Load Balancer)	Receives requests and forwards them to your backend
Spring boot	Processes API requests
RDS MYSQL	Stores your application data

Understanding CURL :

We don't need a browser to become a client and we can always use curl : For example :

```
curl  
https://example.com
```

So here : curl is the client and example.com is the server

Is Postman also a client :

You can use postman if your API is :

```
GET http://localhost:8080/api/bills/101
```

So postman is just another HTTP client and you can test APIs without building the frontend first

Clients and Server WHO ? :

Tool	Role
Chrome	Client
curl	Client
Postman	Client
React frontend	Client
Mobile app	Client
Spring boot	Server
Payment API	Server

HTTP Methods :

GET	Give me something	GET /api/bills/101
POST	Create something	POST /api/bills
PUT	Update something	PUT /api/bills/101
DELETE	Delete something	DELETE /api/bills/101

Summary : [Client type : Scenario : Server Sees] :

Client type	Scenario	Server Sees
Browser	Person reading news online	HTTP request from User-Agent : Chrome
Mobile App	Tapping refresh in facebook app	HTTP request from User Agent : IOS APP
Server	Online store sending an email via gmail	HTTP request from your servers backend
Terminal / VSC / CMD	Testing an API with curl or postman	HTTP request from User agent : curl

Important Rule :

Client	Hey server give me /users
Server	Here are the users
Client	Hey server , POST / users with this new user
Server	Ok I added this and here is the final

2. The Shape Of A Real System:

Concept	Very easy meaning	Real-life example	Technical example	Why we use it
DNS	Converts a website name into a destination/address	Phone contacts: Akash → phone number	api.example.com → ALB	Humans remember names; computers need network destinations
Load Balancer	Distributes incoming requests between servers	Traffic police directing cars to different roads	ALB → EC2 #1 / EC2 #2	Prevents one server from getting overloaded
App Server	Machine that runs your application code	Worker doing the actual job	EC2 + Docker + Spring Boot	Processes user requests
Docker	Packages your application so it runs consistently	A standardized box containing everything needed	electricity-backend container	Makes application deployment easier and consistent
Database	Stores structured information	Filing cabinet with organized records	Amazon RDS + MySQL	Stores users, bills, payments, etc.
Object Storage	Stores files/objects	Huge warehouse for documents	Amazon S3	Stores PDFs, images, videos, backups, etc.
Horizontal Scaling	Add more servers	Hire more workers	1 EC2 → 3 EC2 → 10 EC2	Handles increasing traffic
Vertical Scaling	Make one server more powerful	Give one worker a more powerful machine	Small EC2 → larger EC2	Handles more workload on one machine
Health Check	Checks whether a server is working	Calling a worker to	ALB → /health	Prevents traffic from going to broken servers

Concept	Very easy meaning	Real-life example	Technical example	Why we use it
		see if they're available		
High Availability	Application continues working even if one server fails	Having multiple workers instead of depending on one	EC2 #1 fails → EC2 #2 handles traffic	Reduces downtime
Shared Storage	Data accessible by multiple servers	Common company storage room	S3 / RDS	Prevents important data being trapped on one EC2

AWS Service	Job	Remember
Route 53	DNS	Phone Book
Cloudfront	Delivers frontend content globally	Fast delivery network
S3	Stores React Build files	File warehouse
ALB	Distributes API traffic	Traffic controller
EC2	Runs application	Computer / Server
Docker	Packages your application	Application container
RDS	Managed database	Database
MYSQL	Stores structured data	Tables
Auto Scaling	Add remove EC2 servers	Hire / Release workers

Complete Work Flow :

When someone opens your application :

USER

|

| "I want electricity bill"



DNS

|

| "Where is the application?"



LOAD BALANCER

|

| "Which server should handle this?"



EC2 + SPRING BOOT

|

| "I need the bill information."



RDS + MYSQL

|

| "Here is the bill data."



SPRING BOOT

|

| "I need the PDF."



S3

|

| "Here is bill-101.pdf"



SPRING BOOT

|



LOAD BALANCER

|



USER

Important terms :

Client	Users browser or mobile app
Request	User asking the server to do something
Response	Servers answer
Server	Computer that provides a service
App server	Server running your application
Database	Stores structured data
Object storage	Stores file or objects
DNS	Name = destination
Load balancer	Distribute requests
Horizontal scaling	Add servers
Vertical scaling	Make server bigger
Health check	Check if server is working
High availability	Keep application available despite failures
Shared storage	Storage accessible to multiple servers
State	Information that must remembered between request
Stateless App	Any healthy server can handle a request
Durable storage	Data survives server failure

3 . What actually runs on a server : From source code till physical server : Code to hardware : Server stack :

Layer no	Layer name	Meaning	Example	Think
1	Your program	Code written by you	Controller.java Service.java , Bill.java	Our instructions
2	Libraries	Ready made code written by other developers	MYSQL Connector	Ready made tools
3	Framework	Helps you build application without doing everything yourself	Spring Boot	Building framework
4	Build tool	Builds packages your application	Maven : mvn clean package	Pack
5	Compiled code	Java source	.java / .jar	Instructions JVM understands
6	Runtime	Software that actually runs your program	JVM	Person JVM understands
7	Operating System	Manages CPU , RAM , files , network etc	Ubuntu Linux on EC2	Building manager
8	Container	Packages your application and required environment	Docker container	Portable box
9	Networking	Allows applications / computers to communicate	IP , Port 8080 , DNS , TCP	Roads between computers
10	Cloud Infrastructure	AWS provides servers and services	EC2 , ALB , RDS , S3 , Cloudfront	Large rented infrastructure

11	Hardware	Physical resources	CPU , RAM , disk , network card	Physical machine
----	----------	--------------------	---------------------------------	------------------

Step by Step Workflow :

Step 1	Write java source code
Step 2	Spring boot provides framework functionality
Step 3	Maven builds your application
Step 4	Java source is complied
Step 5	.jar contains your application and dependencies
Step 6	Java -jar app.jar starts the JVM
Step 7	JVM executes the java
Step 8	Linux manages the java process
Step 9	CPU executes machine instructions
Step 10	RAM stores the running applications and data
Step 11	Network allows users to send requests
Step 12	Spring boot receives the HTTP request
Step 13	Controller handles /bills
Step 14	Application talks to MYSQL
Step 15	Response goes back to the user

Based on AWS terms :

Component	Job	Meaning
React	Frontend	What the user sees
S3	Stores frontend files	Online storage for website files
Cloudfront	Delivers frontend quickly	Fast delivery
ALB	Receives routes requests	Traffic controller
EC2	Runs backend	Your Virtual Server
Docker	Runs package backend	Application container
JVM	Runs java	Java runtime
Spring boot	Runs your backend logic	Application framework
RDS MYSQL	Stores data	Database
DNS	Converts name to IP	Internet phonebook

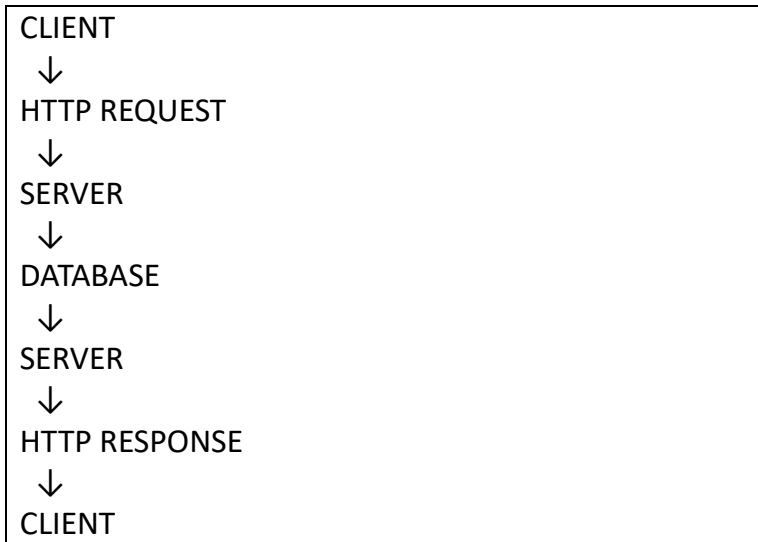
IP address	Identifies machine / network endpoint	Computer's address
Port 8080	Identifies application service	Door to your backend

Source code	Code humans write
Compiler	Converts source into another executable form
Runtime	Software that runs your program
JVM	Runtime for Java
Framework	Structured tools for building applications
Process	Running Program
OS	Manages computer resources
Docker	Packages runs application in containers
Server	Computer provides services to other computers

4. HTTP, Line By Line :

Concepts	Meaning	Example
Client	Asking for something	React frontend / Browser
Server	Receiving the request and doing the work	Spring boot
HTTP	Rules or language used between client and server	React <-> Spring boot
Request	Message sent by client	Give me bill
Response	Messge sent by server	Here is the bill
API	Set of endpoints which applications communicate	/api/bills
Database	Stores application data	MYSQL / RDS
JSON	Common format for sending data	{ "name": Akash" }

Remember the workflow :



HTTP Request :

Concept	Meaning	Example
Request line	What you want to do	POST /api/users
Headers	Extra information	Content-Type: application/json
Body	Actual data being send	{ " name": Akash"}

Request Line : [Example]

POST /api/users HTTP/1.1

Concept	Meaning	Example
POST	HTTP method	What to do you want to ?
/api/users	Path/endpoint	Where and what resource
HTTP/1.1	HTTP version	Which HTTP version

HTTP Methods :

Example:

Request	Meaning
GET /api/bills/101	Give me bill 101
POST /api/customers	Create a customer
PUT /api/customers/42	Update customer 42
PATCH /api/customers/42	Partially update customer 42
DELETE /api/customers/42	Delete customer 42

Method	Meaning	Example
GET	Give me something	Get a bill
POST	Create something	Create a customer
PUT	Replace or update something	Update customer
PATCH	Partially update something	Change only phone number
DELETE	Remove something	Delete customer

Path / Endpoint :

GET /api/bills/101

Concept	Meaning
/api	API area
/bills	Bill resource
/101	Specific bill

Headers :

Headers contain extra information about the request or response

Header	Meaning	Example
Host	Which server or website	Host: example.com
Content-Type	What type of data is being sent	application/json

Content-Type :

Example :

```
Content-Type: application/json
```

This means I am sending data in JSON and spring boot sees it should treat the body as JSON

```
{  
  "name": "Akash",  
  "number": "123456"  
}
```

HTTP Body :

Body contains the actual data

```
POST /api/customers  
Content-Type: application/json  
  
{  
  "name": "Akash",  
  "address": "Coimbatore",  
  "number": "123456"  
}
```

Part

Meaning

Headers	Information about the data
Body	Actual data

Therefore , Header is the information about the message and Body is the actual message or data

HTTP Response :

Server sends a response back

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/customers/42

{
  "id": 42,
  "name": "Akash"
}
```

Part

Meaning

Status line	What happened
Headers	Extra information
Body	Actual result or data

Status Line :

```
HTTP/1.1 201 Created
```

Part

Meaning

HTTP/1.1	HTTP version
201	Status code
Created	Human Readable explanation

Important HTTP Status Codes:

Code	Name	Meaning
200	OK	Everything worked
201	Created	Something was successfully created
204	No content	Worked but nothing to return
400	Bad request	Client sent invalid data
401	Unauthorized	Authentication missing or invalid
403	Forbidden	You are not allowed
404	Not found	Resource doesn't exist
500	Internal Server Error	Server had a problem
503	Service Unavailable	Service / Server unavailable

Response Body :

Data given :

```
{  
  "id": 101,  
  "amount": 1250,  
  "status": "UNPAID"  
}
```

React receives it and displays :

```
Bill #101  
Amount: ₹1250  
Status: UNPAID
```

Location Header :

Example :

Created something is just created is available here :

Location: /api/customers/42

You create	Akash
Server creates	ID = 42

Therefore :

/api/customers/42

Stateless HTTP :

Server does not automatically remember old HTTP requests :

Image :

Request 1 "I am Akash." Request 2 "Give me my bills." Request 3 "Give me my payments."

With stateless communication each request needs the information required to process it

Stateless in AWS :

Image having two EC2 servers :

Request 1 :

Request 2 :

User → ALB → EC2 #1	User → ALB → EC2 #2
---------------------	---------------------

If the application is stateless and EC2 2 can process the second request without depending on the first EC2 1 and this very useful for scaling

So therefore,

HTTP = communication rules between client and server.

Request = Client → Server.

Response = Server → Client.

GET/POST/PUT/PATCH/DELETE tell the server what you want to do.

Headers give extra information, body carries data, and status codes tell you what happened.

5. HTTP Methods :

Concept	Meaning	Example
HTTP	Language or rules used for communication between client and server	React talks to spring boot using HTTP
HTTP Request	Client asking the server to do something	React asks GET /customers/101
HTTP Response	Server's answer	Spring boot returns customer 101
HTTP Method	Tells the server what you want to do	GET reads and POST create
Client	Making the request	Browser/React
Server	Receiving and processing request	Spring boot
Database	Stores application data	MYSQL / RDS Amazon
Resource	Piece of data managed by the API	Customer bill, payment and user

GET :

Meaning	Give me data
Main purpose	Read
Example	GET /customers/101
DB Operation	SELECT

POST :

Meaning	Create something
Purpose	Create resource
Example	POST /customers
DB Operation	Insert

POST : EXAMPLE :

POST /customers

JSON :

```
{  
  "name": "Akash",  
  "city": "Dubai"  
}
```

In Database :

101 Rahul
102 Ashwin
103 Akash ← NEW

PUT :

Meaning	Replace the resource
Purpose	Full Update / Replacement
Example	PUT /customers/101

PUT : EXAMPLE :

PUT /customers/101

JSON

```
{  
  "name": "Akash",  
  "city": "Dubai",  
  "phone": "123456"  
}
```

JSON

```
{  
  "name": "Akash Kumar",  
  "city": "Abu Dhabi",  
  "phone": "999999"  
}
```

PATCH : PARTIAL UPDATE :

Meaning	Change part of something
Purpose	Partial update
Example	PATCH /customers/101

PATCH : EXAMPLE :

PATCH /customers/101

<pre>{ "name": "Akash", "city": "Dubai", "phone": "123456" }</pre>	<pre>{ "phone": "999999" }</pre>
--	------------------------------------

PUT VS PATCH :**PUT :****PATCH**

Replace	Partially modify
Sends full complete	Sends only changed fields
Full update	Partial Update
PUT /customers/101	PATCH /customers/101
Replace	Change Part
Name = Akash City = Dubai Phone = 123456 "Here is the complete new customer."	"Just change the phone number."

DELETE - REMOVE : EXAMPLE :

101 Akash
102 Rahul
103 Ashwin

USING :

DELETE /customers/101

AFTER :

102 Rahul
103 Ashwin

Remember :

GET	→	READ
POST	→	CREATE
PUT	→	REPLACE
PATCH	→	PARTIAL UPDATE
DELETE	→	REMOVE
HEAD	→	HEADERS
OPTIONS	→	WHAT IS ALLOWED?

CRUD AND HTTP Methods :

CRUD	Meaning	HTTP Method
C - Create	Create data	POST
R- Read	Read data	GET
U - Update	Update data	PUT / PATCH
D - Delete	Delete data	DELETE

Method	Remember	CRUD	Example
GET	READ	R	GET /customers/101
POST	CREATE	C	POST /customers
PUT	REPLACE	U	PUT /customers/101
PATCH	PARTIAL CHANGE	U	PATCH /customers/101
DELETE	REMOVE	D	DELETE /customers/101
HEAD	HEADERS	-	HEAD /customers/101
OPTIONS	ALLOWED METHODS	-	OPTIONS /customers

6. Status Code :

When you open a website or use an API , the server sends an HTTP status code with its response

Code	Category	Meaning
2xx	Success	Request worked
3xx	Redirection	Go somewhere or use something else
4xx	Client Error	Your request has a problem
5xx	Server Error	Server has a problem

2xx : Success :

Code	Name	Meaning	Example	Technical
200	OK	Everything worked	Your order pizza arrives	GET /users → 200 OK
201	Created	Something new was created	You make new booking	POST /users → 201 Created
204	No content	Worked but nothing to return	Delete an old item	DELETE /users/10 → 204

3xx : Redirection :

Code	Name	Meaning	Real Life	Technical
301	Moved permanently	Permanently moved	Shop permanently changed address	Old to New URL
302	Found / Temporary Redirect	Temporarily go somewhere else	Restaurant temporarily moved	/login → /temporary-login
304	Not modified	Use your copy	Documents not changed and use the copy	Browser uses cached image

4xx : Client Errors :

Generally means there is something wrong with the request from the client side

Client could be :

Browser : Mobile application : Postman : Frontend React Application : API

400 : Bad Request :

Code	400
Meaning	Request is invalid
Think	I don't understand your request

Example :

Missing comma : Send Broken JSON : Server : 400 Bad Request

```
{  
  "name": "Akash"  
  "age": 25  
}
```

401 : Unauthorized :

Code	401
Meaning	Authentication is missing or invalid
Think	Who are you

Real life :

Security	Show me your ID
You	I don't have one
Security	I cant verify you

403 : Forbidden :

Code	403
Meaning	Authenticated but don't have permission
Think	I know you but NO

Real life :

Security	I know who you are
You	Can I enter the CEO room
Security	No you are not allowed

404 : Not found :

Code	404
Meaning	Requested resource doesn't exist
Think	I cant find it

409 : Conflict :

Code	409
Meaning	Request conflicts with current state
Think	The conflicts with something that already exists

Real lie :

There is one seat	A1
Person 1 books it	A1 booked
You try to book A1	409 Conflict

422 : UnProcessable Content :

Code	422
Meaning	Server understands the request but data fails validation or business rules
Think	I understand you , your data is not acceptable

Example :

You send : The JSON is valid : Server understands it but age is invalid according to application rules

```
{  
  "name": "Akash",  
  "age": -50  
}
```

5xx : Server Errors :

500 : Internal Server Error:

Code	500
Think	Something broke inside my application

502 : Bad Gateway :

Cant get proper backend response

503 : Service Unavailable

Based on AWS :

200 Request worked

201 Resource was created

400 Check request/body/parameters

401 Check login/token

403 Check permissions/roles/CORS/security

404 Check URL/API endpoint/resource

409 Check duplicate/conflicting data

422 Check validation/data

500 Check Spring Boot/application logs

502 Check ALB/Nginx → backend connection

503 Check whether backend/target is healthy and available

7. URLs :

Sno	Concept	Meaning	Example
1	URL	Complete address used to reach something on the internet	https://api.example.com
2	Scheme or Protocol	Tells how to communicate	https://
3	Host / Domain	Tells which server to contact	api.example.com
4	Port	Tells which network service to use	:443
5	Path	Tells which resource or data you want	https://api.example.com/users/42
6	Path Parameter	Value inside the path that identify a specific resource	42
7	Query String	Starts after ? : Tells the server how you want the data	?status=open&limit=10
8	Query parameter	One individual option inside the query string	status=open

With a Complete URL :

<code>https://api.example.com:443/users/42/orders?status=open&limit=10#section</code>

Part	Value	Meaning
Scheme	https	How to communicate
Host	api.example.com	Which server
Port	443	Which network door
Path	/users/42/orders	Which resource
Path parameter	42	Which specific user
Query String	?status=open&limit=10	How to filter and modify result
Query parameter 1	status=open	Only open orders
Query parameter 2	limit=10	Max 1 results
Fragment	#section	Particular page section

8 · HTTPS and TLS :

Concept	Meaning	Technical	Example
HTTP	Language used by browser and server	Protocol for web requests and responses	-
HTTPS	Secure HTTP	HTTP running over TLS	https://google.com
TLS	Security and protection layer	Protocol secures communication	Encrypts and protects HTTP traffic
SSL	Old name	Replaced by TLS	SSL certificate
Certificate	Website digital ID card	Digital certificate binding a domain	Certificate for example.com
CA	Trusted organization that verifies certificates	Certificate Authority	Encrypt Digital Certificate
Encryption	Scrambles information	Converts readable data into protected ciphertext	Password123 – unreadable encrypted data
Decryption	Turns protected data back into usable data	-	Server processes protected request

HTTP VS HTTPS :

HTTP	HTTPS
http://example.com	https://example.com
No TLS Security	Uses TLS
Data not protected by TLS	Data is encrypted and protected
No TLS certificate authentication	TLS protects against these attacks
Easier to intercept read and modify traffic	TLS protects against these attacks
Not good for sensitive applications	Standard choice for applications

Problems does HTTPS and HTTP solve :

Imagine you are using public Wi-Fi :

Without HTTPS Someone monitoring the network could inspect unprotected traffic Your Browser ↓ Public Wi-Fi ↓ Internet ↓ Server	With HTTPS TLS protects the contents of the connection while they travel between the endpoints Your Browser ↓ TLS protection ↓ Public Wi-Fi ↓ TLS protection ↓ Internet ↓ TLS protection ↓ Server
---	--

Things that TLS Provides :

Feature	Meaning	Example	Technical
Confidentially	Keep information secret	Nobody should read your letter	Password is not sent as readable plaintext
Integrity	Make sure information wasn't secretly changed	Nobody changes 100rs to 10,000 in your letter	TLS detects tampering
Authentication	Verify who you are talking to	Check someone ID card	Certificate helps verify the server or domain

TLS Handshake :

Happens when the browser and server establish the secure connection

It like :

Hello : Prove who you are : Agree on security : Establish keys : Start secure communication

Step	Happens	Meaning
1	ClientHello	Browser says hello
2	ServerHello	Server responds
3	Certificate	Server shows digital ID
4	Browser verifies certificate	Browser checks server identity
5	Key exchange	They establish cryptographic secrets
6	Secure session	Protected communication begins
7	HTTP traffic	Browser server exchange protected application data

Ports :

Protocol	Common port
HTTP	80
HTTPS	443
SSH	22
Spring boot	8080
MYSQL	3306

Padlock :

If your browser shows https://example.com and it generally means the browser has established a TLS connection and successfully performed its certificate and security checks

But doesn't know mean HTTPS is definitely honest and mean something closer to be same or identical with same authenticating and certificate checks and protection

9 . DNS and ports :

Client	Who is asking
Server	Who is answering
Request	What is being asked
Response	What is being returned
IP	Where is the machine
DNS	How do I find the machine or destination by name
Port	Which services should receive the request
HTTP /HTTPS	How do web clients communicate
Docker	How do I package or run my application
AWS architecture	Where does each component run

Client	Person asking	Browser
Server	Computer answering	EC2
Request	Question	GET /biils
Response	Answer	JSON
Domain	Human friendly name	Example.com
DNS	Name for destination	Domain
IP	Network address	54.123.45.67
Port	Service to door	8080
HTTP	Web communication	:80
HTTPS	Secure Web communication	:443
SSH	Remote login	:22
MYSQL	Database	:3306
React	Frontend	:3000 locally
Spring Boot	Backend	:8080
Docker	Container application	Container
EC2	Virtual server	AWS Computer
S3	File or object storage	React files
Cloudfront	-	Fast frontend delivery
ALB	Load balancer	Routes traffic
RDS	Managed database	MYSQL
ECR	Docker image registry	Stores images

Security Groups:

Acts like a security guard or firewall and open the ports that are actually need to communicate

Server	Port	Allow From	WHY
EC2	22	Your IP	SSH
ALB	443	Internet	HTTPS
ALB	80	Internet	HTTP
EC2	8080	ALB	Spring boot
RDS	3306	EC2	MYSQL

AWS Services :

Service	Job	Meaning
Route 53 / DNS	Domain routing	Internet phone book
Cloudfront	Frontend delivery	Fast global delivery
S3	Stores react files	Online storage
ALB	Sends request to backend	Traffic controller
EC2	Runs backend	Virtual computer
Docker	Packages or runs application	Application container
ECR	Stores docker images	Docker image storage
RDS	Runs MYSQL database	Managed database

Technology and Layer :

Layer	Technology
Frontend	React
Backend	Spring boot
Database	MYSQL
Container	Docker
Cloud	AWS

Docker : Ports :

Suppose spring boot application runs inside a docker container:

Inside the container :

```
Spring Boot : 8080
```

You run :

```
docker run -p 8080:8080 electricity-backend  
or  
9000:8080
```

Format :

```
docker run -p
```

```
-p HOST_PORT:CONTAINER_PORT
```

Therefore : 8080 : 8080 [Host port : Container port]

9000 : 8080 [Host : Container] [Access localhost : 9000] [Outside : Inside]

Port :

A port is like a door on a machine and suppose : IP : 54.123.45.67 and that same machine could have :

```
54.123.45.67:22  
54.123.45.67:80  
54.123.45.67:443  
54.123.45.67:8080
```

IP : Address / Port : Number / Service : Working behind the port

Suppose :

54.123.45.67:8080

This means go to machine 54.123.45.67:8080 and communicate with the service port 8080

Port	Used for	Meaning
22	SSH	Remote server login
80	HTTP	Normal Web traffic
443	HTTPS	Secure web traffic
3306	MYSQL	MYSQL Database
8080	Applications	Backend / app port
3000	React	Development port

Port 22 : SSH

SSH allows you to remotely access a server while connecting EC2 server using SSH with port 22 is commonly done

Port 80 : HTTP

Normal web communication where HTTP = 80 is normal web number

Port 443 : HTTPS

Secure web communication and where 443 is secure number

Port 3306 : MYSQL

MYSQL commonly uses 3306

Port 8080 : Spring boot

Application uses : server.port=8080 and application listens on 8080 and can access it locally using `http://localhost:8080` [localhost : Computer / 8080 : Spring boot application]

Port 3000 : React

Developments server uses 3000 so `http://localhost:300` means computer uses port 3000 to connect react server

Can have both running parallel : `localhost:3000 /React`

`localhost:8080 / Spring Boot`